

Quarantine Relief work Optimisation in Novella City

Task 1: Dynamic Programming

a) Using a 5-minute limit, comment on the sizes of problems that could be solved

To accurately determine what can be done in 5-minute limit using the multiple-choice knapsack simulation, numerous tests involving variables of different sizes were conducted. The results which I have found is based on my solution which is heavily influenced by the prog05KnapsackProblem.

```
problemGenerator p = new problemGenerator(1000, 6, 5, 20, 20,  
System.nanoTime());
```

Using this problem as the base case it measured to 3,635,281,600ns (3.65sec) to solve, using this observation we can tell that machine can handle a lot more people, or so it would seem.

However, when testing using the maximum amount as listed by the comments within the code, it leads to out of memory error. This continues to be the case till its tested with around 2000 people where it seems to be generating a time of 9 seconds, depending on the seed. This is most likely due to the exponential nature of dynamic programming; the memory is $O(\text{numItem} \times \text{budget})$.

If the program was optimised, using the base line of 3.65 secs for 1000 people, then in a 5-minute window it can potentially solve an issue involving 82,417, note that this is with other variables maxed out.

b) Discuss how to improve the time and memory performance of your program further

- Reducing memory usage, currently for the dynamic programming problem it uses a two dimensional array, this is the main reason for the crazy memory usage ($\text{numItem} \times \text{budget}$), potentially it can be modified to use a 1D array, and instead of storing all numItem rows, only store one at a time, this will reduce memory to $O(\text{budget})$.
- Currently my program reconstructs the problem at the end so it can calculate budget costs, doing this on the fly (perhaps after a candidate has been confirmed) can reduce the memory.
- One of the observations I've made with the random seeds is sometimes the options suck, high-cost low value, it might be possible to ignore these entirely, reducing options for categories and therefore memory.
- During some research on solutions of large knapsack problems, the concept of scaling was presented, which in this case seems to mean dividing the budget and cost by a consistent factor, which will reduce the DP array size, but this will also mean the answer is slightly approximated.

Task 2: Backtracking and BnB

c) Running experiments for MCKP and comparing backtracking with or without BnB

I conducted numerous tests, comparing the plain multiple-choice knapsack with knapsack with both backtracking and branch and bound method implemented. To

accurately and efficiently measure the times for each method, and easily test I created used:

```
long start = System.nanoTime();
KnapsackProblem.knapsackSolution(p);
long end = System.nanoTime();
double elapsedSec = (end - start) / 1e9;
System.out.println("Elapsed time: " + elapsedSec + " sec");
long start2 = System.nanoTime();
KnapsackBnB.solve(p); // or backtracking without BnB
long end2 = System.nanoTime();
double elapsedSec2 = (end2 - start2) / 1e9;
System.out.println("Elapsed time BNB: " + elapsedSec2 + " sec");
// keep track of the clock to measure time
long endTime = System.nanoTime();
```

This print outed the exact times for each test.

Test 1: Low quantity

After running the program five different times using these settings

problemGenerator p = new problemGenerator(3, 3, 3, 20, 20, System.nanoTime());

Instance	Optimal Value (No BnB)	Elapsed Time (No BnB, sec)	Optimal Value (BnB)	Elapsed Time (BnB, sec)
1	142	0.0006802	142	0.0009402
2	101	0.0006384	101	0.0012514
3	114	0.0007095	114	0.0010217
4	111	0.0008127	111	0.0010691
5	133	0.0006939	133	0.0009284

For a smaller scale problem such as this one BNB is slightly slower than no BNB, this is as expected since computing the bound and does take a little longer than just exploring all the nodes, the gain from computing the bound isn't fully realized on these small-scale problems.

Test 2: Higher values

This test uses slightly higher values to test effectiveness of pruning, although ideally these values would be much higher to better showcase BNB, unfortunately there

seems to be an issue with my code, where any higher causes it to surpass 5-minute limit.

```
problemGenerator p = new problemGenerator(9, 3, 3, 10, 10, System.nanoTime());
```

Using these parameters, I got these results

Instance	Optimal Value (No BnB)	Elapsed Time (No BnB, sec)	Optimal Value (BnB)	Elapsed Time (BnB, sec)
1	181	0.0009803	181	0.3502925
2	155	0.001079	101	19.726

As you can see, they are heavily varied, but I believe they do make the best case for BNB. Instance one is where the pruning was effective and good result was achieved because of it, and in instance two the pruning proved to be ineffective and made the performance significantly worse. If we were to consider these times for significantly bigger problems, the time saved by pruning would be exponential, and loss would become less noticeable.